Reproducing FinMem on a Leakage-Free Window

Implementation Development Report

MBA Data-Science Seminar — reproduction & critical assessment of
Yu et al., *FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design* (arXiv:2311.13743)

Dan Shoshan & Nimrod Sagi

Code: github.com/Doubledanx2/seminarion_DS_Finmem_implementation

1. Executive summary

We reproduced the FinMem LLM trading agent on the authors' own codebase and re-ran the entire experiment on a **leakage-free 2026 test window** (2026-01-02 → 2026-06-01) that postdates every model's knowledge cutoff — the fix for the paper's central flaw, where its 2022–23 test window sat inside the backbone LLM's training data. We built the full data pipeline (Alpaca news, SEC filings, Gemini summarisation, FinBERT sentiment), fixed numerous bugs in the research code, assembled a corrected and extended configuration we call **FinMem-Ours**, and benchmarked it against three baselines under one audited, canonical accounting convention.

Headline result. On out-of-sample data the elaborate layered-memory + persona apparatus did not add value — it was beaten by simpler baselines:

Strategy	Mean cum. return (0 bps)	Mean Sharpe
No-memory ablation	+1.7%	+0.38
LC-Trader (plain long-context)	−2.9%	+0.16
Buy & Hold	−4.1%	+0.11
FinMem-Ours (full apparatus)	−5.3%	−0.15

Both stripped-down baselines — removing memory (the ablation) and a plain long-context model fed the same news (LC-Trader) — outperformed the complete system, which even trailed passive Buy & Hold. This is the cleanest statement of our critical assessment.

2. Objective & experimental design

Leakage thesis (Backtesting Sin #2). The paper's reported out-performance may reflect the GPT-4-Turbo backbone *remembering* 2022–23 prices rather than predicting them. Our design forces every generative component to have a knowledge cutoff before the data window: decision model **gpt-4.1-mini** (cutoff Jun-2024), summariser **Gemini 3.1 Flash-Lite** (Jan-2025); train 2025-07-01→12-31, test 2026-01-02→06-01.

Tickers: TSLA, NFLX, AMZN, MSFT, COIN (the paper's five). **Hyper-parameters frozen** at the paper's published values before any test run (git-hash recorded) to avoid data-snooping (Sin #4). Comparisons pre-declared: FinMem-Ours vs Buy&Hold, vs no-memory, vs LC-Trader, vs the as-shipped artefact.

3. Data pipeline

News: Alpaca historical news API, 18,311 articles across the 5 tickers (full monthly coverage; single-source Benzinga feed — disclosed limitation). **Filings:** SEC 10-K/10-Q MD&A via sec-api.io (31 credits) keyed by *filedAt* (publication date), never period-of-report. **Summarisation:** all news + filings condensed by Gemini 3.1 Flash-Lite with strict per-article isolation (no cross-article or background context). **Sentiment:** FinBERT (yiyanghkust/finbert-tone) locally on an RTX 3090. **Prices:** yfinance adjusted close. Output: one per-ticker model-input pickle stepped through the paper's MarketEnvironment.

4. Implementation challenges (selected bugs in the research code)

ID	Issue found in the shipped code	Resolution
B7	FinBERT label order is {0:Neutral,1:Positive,2:Negative} but the code read po[illegible] — the paper fed P(Negative) as the 'positive'	[illegible]
B8	The self-adaptive risk persona is NOT implemented — only a static one-sided [illegible] rule is commente[illegible]	[illegible] (3-day CR sign)
B14	Empty-news days push an empty list to FAISS → crash (news != {} is alwa[illegible]	Truthiness gate
B16	trading_reflection swallowed exceptions, returning {} silently — days va[illegible]	[illegible] explicit hold fallback
B20	Our OWN metrics used raw decisions as the position (hold=flat, sell=SHORT) [illegible] — inflated every cell	Canonical accounting (see [illegible])
pin	Pre-1.0 OpenAI API, hard-coded Linux paths, guardrails-ai pinned to Python [illegible] removed by yfinance	[illegible]

Every issue is catalogued with IDs B1–B20 / D1–D35 in IMPLEMENTATION_LOG.md (in the code zip).

5. FinMem-Ours — the corrected & extended configuration

FinMem-Ours is the paper's architecture plus our fixes, frozen at one git hash. Key elements:

• **Self-adaptive persona** (paper_rule): risk-seeking/averse by the sign of the 3-day cumulative return (paper §3.1), reconstructing the commented-out rule.
• **Deep-memory retention fix:** the as-shipped deep layer is a 3-day revolving door (entry bar == exit bar + decay → nothing persists, no filing ever retained). We disable downward jumps and use pure age-based recency, so the deep layer actually retains knowledge.
• **Extended reflection** (paper-described but never shipped): a daily M-day self-review synthesised into deep memory.
• **Filing seeding:** the most recent 10-K/10-Q as of day-1 are ingested with true filedAt dates (fixes a boundary gap where pre-window filings were never seen).
• **Long-only unit positions {0,+1}** (the shipped portfolio allowed costless shorting — Sin #7); ada-002 embeddings; observation = 7-day cumulative return.

6. Backtest integrity & the metrics self-audit

We treated the 'Seven Sins of Quantitative Investing' as a test-suite. Beyond the leakage fix (Sin #2) and frozen hyper-parameters (Sin #4), the most important episode was a **self-audit (Sin #5)**: an independent recompute disagreed with our committed numbers. We found two bugs in our own reporting — (1) the return series dropped the final test day (a –4.57% TSLA day), and (2) it treated raw decisions as the position, implicitly shorting on every 'sell'. Both inflated results (e.g. NFLX no-memory read +57% vs the true +19.8%). We adopted ONE canonical convention — **carry-forward unit long-only positions, simple compounded returns, full price series including the terminal day** — with a regression assertion (B&H must equal P[last]/P[first]–1) that fails the run otherwise. Every figure below uses it. Catching our own unaudited-backtest bug is itself a result.

7. Results (canonical convention, test 2026 H1)

Ticker	Strategy	CR 0bps	CR 10bps	Sharpe	Sortino	MaxDD	Turn	%Long
TSLA	FinMem-Ours	-23.1%	-25.5%	-1.92	-2.57	-30%	31	54%
TSLA	No-memory	-5.5%	-8.0%	-0.37	-0.47	-26%	27	43%
TSLA	LC-Trader	-7.9%	-8.1%	-0.33	-0.61	-24%	3	98%
TSLA	BuyHold	-5.1%	-5.2%	-0.14	-0.25	-24%	1	100%
TSLA	As-shipped	-9.2%	-10.2%	-0.50	-0.82	-27%	11	80%
NFLX	FinMem-Ours	+3.8%	+1.9%	0.44	0.48	-15%	18	55%
NFLX	No-memory	+19.8%	+17.5%	1.56	1.36	-13%	20	39%
NFLX	LC-Trader	+2.1%	+1.2%	0.32	0.49	-21%	9	96%
NFLX	BuyHold	-5.6%	-5.7%	-0.19	-0.30	-21%	1	100%
AMZN	FinMem-Ours	+15.5%	+13.3%	1.67	2.56	-13%	19	67%
AMZN	No-memory	+24.5%	+22.1%	2.74	4.15	-7%	19	54%
AMZN	LC-Trader	+15.3%	+15.2%	1.31	2.15	-20%	1	100%
AMZN	BuyHold	+15.3%	+15.2%	1.31	2.15	-20%	1	100%
MSFT	FinMem-Ours	-3.9%	-5.5%	-0.20	-0.20	-21%	17	72%
MSFT	No-memory	-6.0%	-8.2%	-0.56	-0.37	-16%	23	42%
MSFT	LC-Trader	-3.6%	-4.1%	-0.12	-0.15	-27%	5	98%
MSFT	BuyHold	-2.2%	-2.3%	-0.00	-0.00	-26%	1	100%
COIN	FinMem-Ours	-18.5%	-20.5%	-0.70	-1.02	-28%	24	57%
COIN	No-memory	-24.2%	-25.9%	-1.49	-1.62	-25%	23	37%
COIN	LC-Trader	-20.7%	-20.9%	-0.35	-0.66	-45%	3	99%
COIN	BuyHold	-22.8%	-22.9%	-0.44	-0.82	-45%	1	100%

Pooled Wilcoxon (daily): FinMem-Ours vs Buy&Hold p=0.69 (n.s.); FinMem-Ours vs No-memory p=0.075 (memory hurt, median –28 bps/day). Costs reported at 0 and 10 bps; a 0–50 bps break-even sweep is in the code.

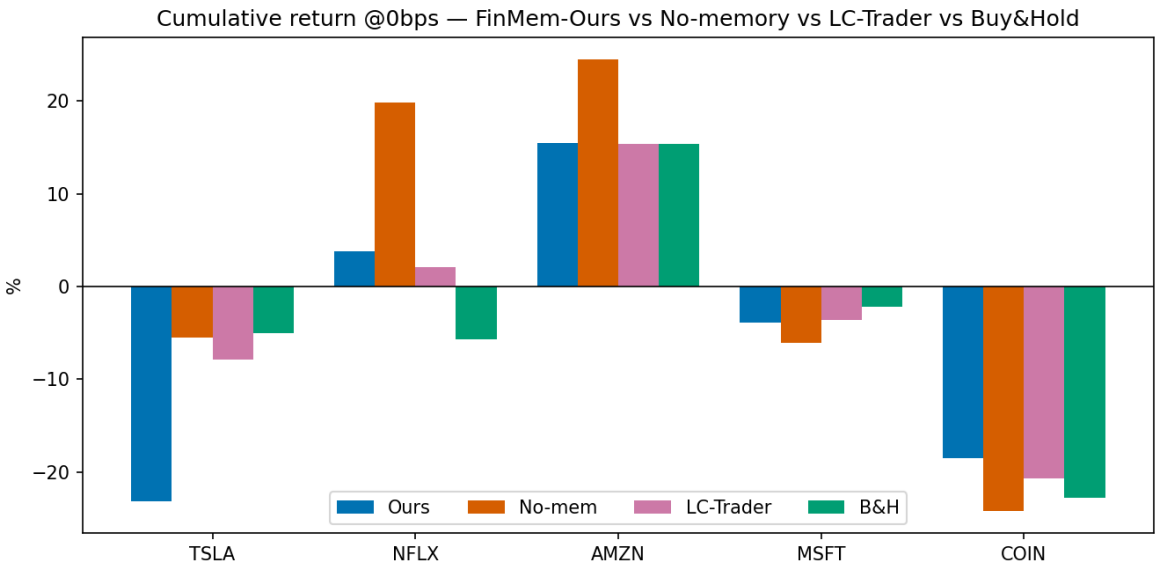

Fig 1. Cumulative return by ticker, all four strategies (0 bps).

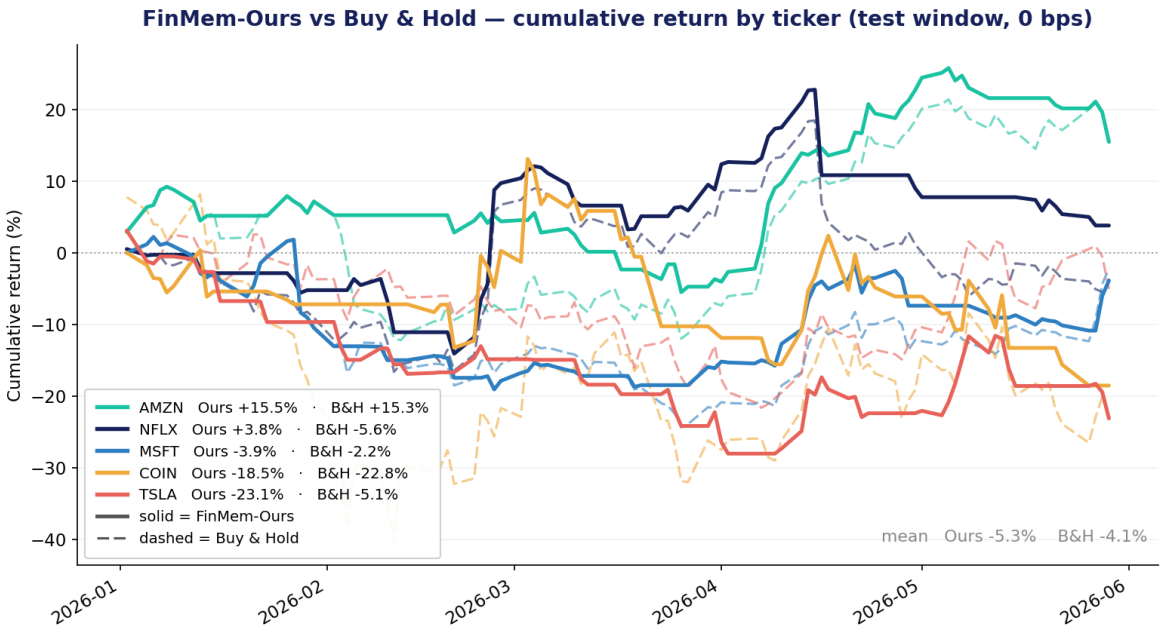

Fig 2. FinMem-Ours (solid) vs Buy & Hold (dashed) per ticker.

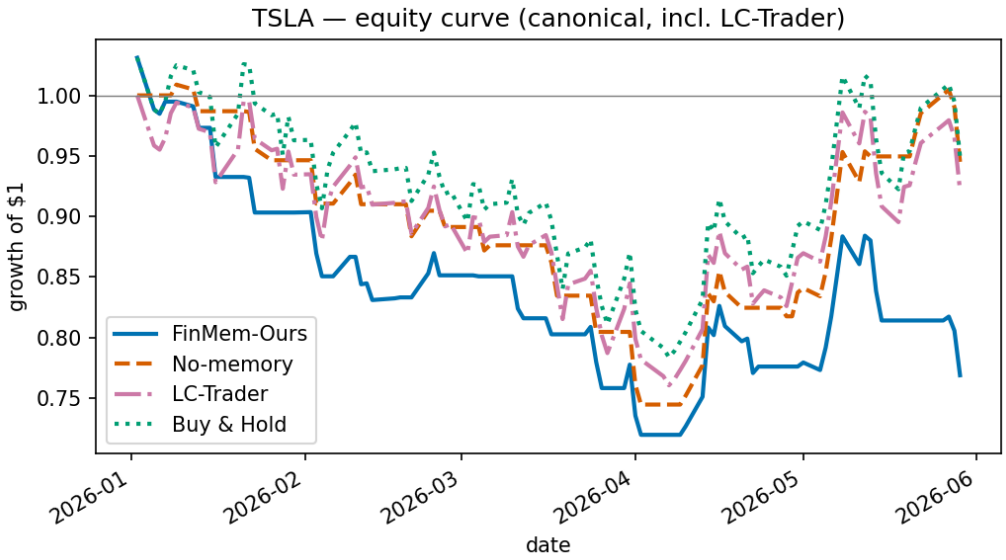

Fig 3. TSLA equity curves — FinMem-Ours is the lowest line.

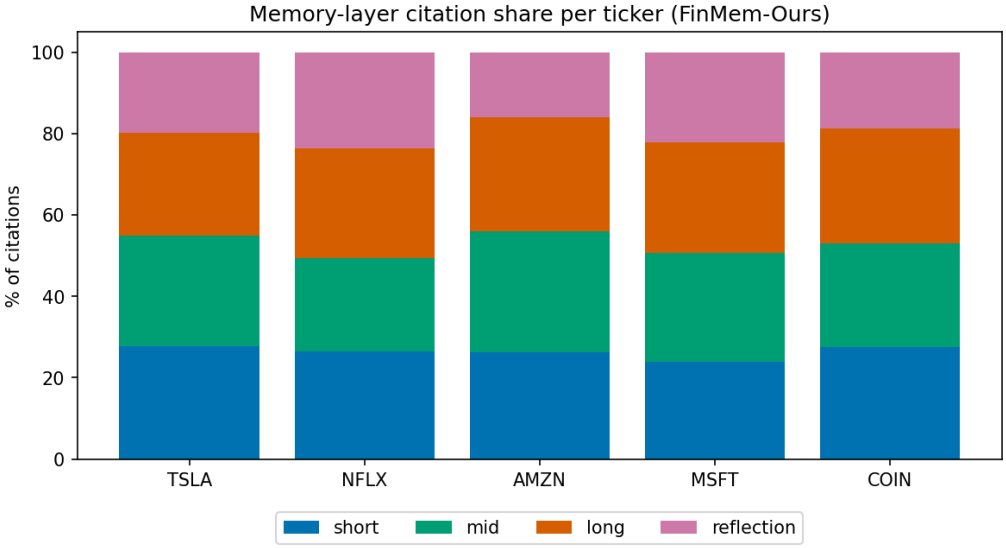

Fig 4. Memory-layer citation share — the agent does use its deep/reflection memory.

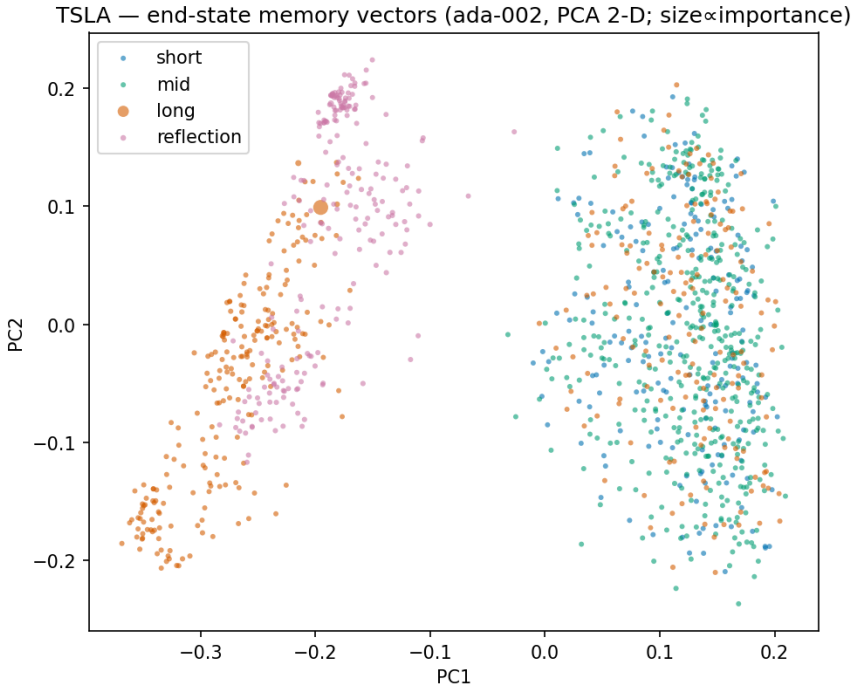

Fig 5. End-state memory vectors (ada-002, PCA), coloured by layer.

8. Key findings

F1 — Pure momentum following. The as-shipped agent agreed with 3-day price momentum on 100% of its trading decisions; our changes moved this to 74%, but performance did not improve.

F2 — Deep memory is a revolving door. Offline replay of the authors' own per-day memory dumps: 176 items entered the deep layer, every one expelled within 3 days, and zero SEC filings were ever retained — the entry threshold equals the exit threshold while importance only decays. The architecture's headline retention claim is not realised by the shipped parameters.

F3 — Memory subtracted value. The no-memory ablation (same backbone, same prompt, empty retrieval) beat the full FinMem-Ours on the mean (+1.7% vs –5.3%) and on 3/5 tickers.

F4 — A plain long-context model is a stronger baseline. LC-Trader — no persona, memory, retrieval, embeddings or sentiment, just the same news streamed into gpt-4.1-mini — was almost entirely passive (≈97% days long), tracked the market, and beat FinMem-Ours on the mean (–2.9% vs –5.3%). Prompt-caching cut its cost 71% ($2.47 for 515 calls), though cost scales quadratically with horizon — a structural overhead the fixed-size memory avoids.

9. Reproducibility

All artefacts are checkpointed and every result is regenerable from saved decisions with $0 spend. Pipeline: `data-pipeline/01..05` (data), `run.py` (train/test), `lc_trader.py` (baseline), `16_canonical_metrics.py` (audited metrics), `17/20/21/22_*.py` (figures), `tests/` (leakage + behaviour suites). Money gates, a token/cost meter with hard caps, and checkpoint-resume across quota resets are built in; the overnight grid even survived a mid-run PC crash with zero data loss.

Appendix — GitHub code vs paper: discrepancies

Aspect	Paper / README implies	Shipped code actually does
Sentiment	FinBERT labels used correctly	Reads P(Negative) as 'positive' (B7)
Persona	Self-adaptive risk persona	Static one-sided line only (B8)
Deep memory	Retains info beyond human limits	3-day revolving door, no filings (F2)
Decay	Q_shallow = 14 (sensitivity-justified)	Ships Q_shallow = 3 (D20)
Shorting	Long-only narrative	'Sell' opens a costless short (Sin #7)
Seeding	Filings inform decisions	Pre-window filings never ingested
Metrics (ours)	—	Raw-decision position + dropped last day (B20)